Introduction to Algorithms
3rd Edition

—

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, & Clifford Stein

Graham Strickland

July 28, 2026

Contents

1 The Role of Algorithms in Computing

1.1 Algorithms

1.1-1

Suppose we need to find the smallest number of buildings in a suburb of a city which need security huts for surveillance of vehicles entering the suburb. We can model this problem by trying to find the convex hull, where buildings are vertices are the n points in the plane.

1.1-2

Space required for data storage, since memory is finite in any computing system executing an algorithm.

1.1-3

Linked lists have the following strengths:

- They are easy to insert items into.
- They are easy to remove items from.
- They do not require a large block of contiguous memore to be allocated ahead of insertion.

They have the following limitations:

- Traversing the linked list is slow.
- Accessing an element is not easy and requires indirect methods.
- Deallocating the memory used is slow.

1.1-4

The traveling-salesman problem can be interpreted as a series of many different shortest-path problems which are combined into one. However, the shortest-path problem does not require that whoever traverses the path return to where they started. It can also be solved efficiently, whereas the traveling-salesman problem cannot.

1.1-5

If we are tring to find the optimal layout of train routes so that trains do not cross tracks at the same time, then only the best solution will do. On the other hand, if we are trying to minimize the cost of procuring parts for the train, then an approximation to the best solution may be good enough.

1.2 Algorithms as a technology

1.2-1

A Google search uses algorithmic content to find matches to a seqrch query in the shortest time possible. The algorithms used must find the shortest path between various networks of web pages as well as parse through large amounts of text data in the shortest time possible.

1.2-2

We are looking for the interval where

$$\begin{aligned} 8n^2 &< 64n \lg n \\ \Rightarrow n &< 8 \lg n \\ \Rightarrow \frac{n}{\lg n} &< 8 \\ \Rightarrow -n \lg n &< 8 \\ \Rightarrow n \lg n &\geq 8. \end{aligned}$$

For $n < 44$, we have insertion sort faster than merge sort.

1.2-3

We are looking for the interval where $100n^2 < 2^n$, thus for $n = 15$, n^2 runs faster than 2^n.

Problems

1-1

From the program in `cpp/compruntime/main.cpp`, we have Table 1.

$f(n)$	1 second	1 minute	1 hour	1 day	1 month	1 year	1 century
$\lg n$	inf	inf	inf	inf	inf	inf	inf
$\sqrt{n}$	1e+12	3.6e+15	1.296e+19	7.46496e+21	6.71846e+24	9.94519e+26	9.94519e+30
n	1e+06	6e+07	3.6e+09	8.64e+10	2.592e+12	3.1536e+13	3.1536e+15
$n \lg n$	62746	2.80142e+06	1.33378e+08	2.75515e+09	7.18709e+10	7.97634e+11	6.8611e+13
n^2	1000	7745	60000	293938	1.60997e+06	5.61569e+06	5.61569e+07
n^3	100	391	1532	4420	13736	31593	146645
2^n	19	25	31	36	41	44	51
$n!$	9	11	12	13	15	16	17

Table 1: Comparison of running times for the largest problem size n of a problem that can be solved in time t, assuming that the algorithm to solve the problem takes $f(n)$ microseconds

In generating the above results, we have made use of the following two C++ functions in `cpp/algorithms/big_oh.cpp`:

```
double inverse_nlogn(double x) {
  std::size_t max_iters = 10;
  double a_0 = x / std::log2(x), a_1 = 0.0;

  for (std::size_t i = 0; i < max_iters; ++i) {
    a_1 = a_0 - ((a_0 * std::log2(a_0) - x) /
                 ((1.0 / std::log(2.0)) + std::log2(a_0)));
    if (std::abs((a_1 * std::log2(a_1)) - (a_0 * std::log2(a_0))) < 1)
      return a_1;
    else
      a_0 = a_1;
  }

  return a_1;
}
```

and:

```
double inverse_factorial(double x) {
  double n = 0, fact = 1;

  while (fact * (n + 1) <= x) {
    n++;
    fact *= n;
  }

  return n;
}
```

`inverse_nlogn` makes use of Newton's method to calculate an approximate value $m(x)$ s.t.

$$n \lg n = x \Rightarrow m(x) \approx n,$$

while `inverse_factorial` generates an approximate value $m(x)$ s.t.

$$x = n! \Rightarrow m(x) = \lfloor n \rfloor.$$

2 Getting Started

2.1 Insertion sort

2.1-1

In Figure 1, we illustrate the operation of INSERTION-SORT on the array $A = \langle 31, 41, 59, 26, 41, 58 \rangle$.

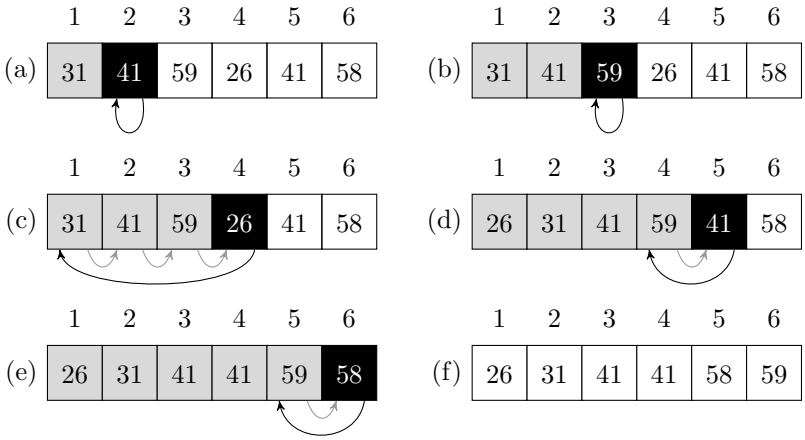

Figure 1: Illustration of INSERTION-SORT.

2.1-2

INSERTION-SORT(A)
1: **for** $j = 2$ **to** $A.length$ **do**
2: $key = A[j]$
3: // Insert $A[j]$ to the sorted sequence $A[1..j-1]$
4: $i = j - 1$
5: **while** $i > 0$ **and** $A[i] < key$ **do**
6: $A[i+1] = A[i]$
7: **if** $i > 0$ **then**
8: $i = i - 1$
9: **else**
10: $i = 0$
11: $A[i+1] = key$

2.1-3

LINEAR-SEARCH(A, ν)
1: $j = 1$
2: **while** $j \neq A.length$ **do**
3: **if** $A[j] == \nu$ **then**
4: **return** j
5: **else**
6: $j = j + 1$
7: **return** NIL

Initialisation: With the loop invariant being that $A[j]$ refers to an element of the sequence $A = \langle a_1, a_2, \ldots, a_n \rangle$ and each element in $A[1..j-1]$ has

been checked for equality, we have the loop invariant valid, since $j = 1$ and $A[1]$ is in A for $n \geq 1$, at the start of the **while** loop 1 – 6.

Maintenance: Either the **if** statement in line 3 returns $A[j]$ or the **else** statement in line 5 increments j by 1, so that after each pass of the **while** loop, we have checked whether or not $A[j] = \nu$, and the condition of the **while** loop ensures that $A[j]$ is in A.

Termination: If the **while** loop terminates, then either the **if** condition in line 3 is true, so that $A[j] = \nu$ or $j = A.length$, at which point we return the special value NIL.

Thus the algorithm is correct, since the loop invariant is initialised and maintained throughout, and the algorithm terminates with the correct output; if a value is returned, ν is in A, and occurs at $A[j]$ for return value j, otherwise the special value NIL was returned, and ν is not in A.

2.1-4

We have the following addition problem:

Input: Two n-element arrays A and B containing binary digits.

Output: $(n + 1)$-element array C containing the sum of A and B.

We have the following algorithm as a solution:

```
BINARY-ADDITION(A, B, n)
 1: carry = 0
 2: for i = 1 to n do
 3:    if A[i] = 1 and B[i] = 1 then
 4:       if carry == 0 then
 5:          C[i] = 0
 6:          carry = 1
 7:       else
 8:          C[i] = 1
 9:          carry = 1
10:    else if A[i] == 1 and B[i] == 0 or
             A[i] == 0 and B[i] == 1 then
11:       if carry == 0 then
12:          C[i] = 1
13:       else
14:          C[i] = 0
15:          carry = 1
16:    else
17:       C[i] = carry
18:       carry = 0
19: C[n + 1] = carry
20: return C
```

2.2 Analyzing algorithms

2.2-1

$$\frac{n^3}{1000} - 1000n^2 - 100n + 3 \approx \Theta(n^3).$$

2.2-2

SELECTION-SORT(A)
1: **for** $i = 1$ **to** $A.length - 1$ **do**
2: $\quad smallest = i$
3: $\quad$ **for** $j = i$ **to** $A.length$ **do**
4: $\quad\quad$ **if** $A[j] < A[smallest]$ **then**
5: $\quad\quad\quad smallest = j$
6: $\quad A[smallest] = A[i]$

The **for** loop from lines 2-8 maintains the loop invariant that $A[1\,..\,i-1]$ contains sorted elements in ascending order.

The algorithm only needs to run for the the first $n-1$ elements since the nth element will already be the largest element in the array after each smaller element in $A[1\,..\,n]$ has been sorted.

Since there is no **while** loop and each search for the smallest item in the subarray $A[i+1\,..\,n-1]$ must occur, the best and wors-case running times are the same, i.e., $\Theta(n^2)$.

2.2-3

Assuming the element being searched for is equally likely to be at any position in the array, we would on average search through

$$\frac{1+2+\cdots+n}{n} = \frac{1}{n}\sum_{k=1}^{n} k = \frac{n+1}{2}$$

elements for an input sequence of length n. In the worse case we would search through all n elements.

Thus the average- and worst-case running times are $\Theta(n)$, since

$$\frac{n+1}{2} = \frac{n}{2} + \frac{1}{2},$$

and the dominant term is $\frac{1}{2}n$, which is equivalent to $\Theta(n)$. Obviously, looping through n items sequentially is $\Theta(n)$, so the average- and worse-case running times are in an equivalent Θ-class.

2.2-4

By only inputting elements that are equivalent or close to the expected output.

2.3 Designing algorithms

2.3-1

In Figure 2, we illustrate the operation of MERGE-SORT on the array $A = \langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$.

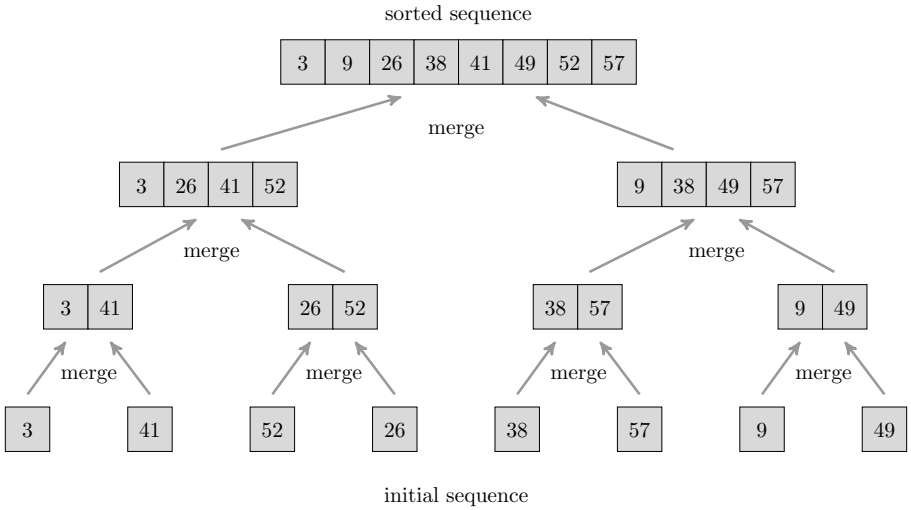

Figure 2: Illustration of MERGE-SORT.

2.3-2

MERGE(A, p, q, r)
1: $n_1 = q - p + 1$
2: $n_2 = r - q$
3: let $L[1 \,..\, n_1 + 1]$ and $R[1 \,..\, n_2 + 1]$ be new arrays
4: **for** $i = 1$ **to** n_1 **do**
5: $\quad L[i] = A[p + i - 1]$
6: **for** $j = 1$ **to** n_2 **do**
7: $\quad R[j] = A[q + j]$
8: $i = 1$
9: $j = 1$
10: **for** $k = p$ **to** r **do**
11: $\quad$ **if** $i \leq n_1 + 1$ and $L[i] \leq R[j]$ **then**
12: $\quad\quad A[k] = L[i]$
13: $\quad\quad i = i + 1$
14: $\quad$ **else if** $A[k] == R[j]$ **then**
15: $\quad\quad j = j + 1$

2.3-3

Proof. We have the base case for $n = 2^1$ given by

$$T(n) = T(2) = 2 = 2\log_2 2^1 = 2 \cdot 1.$$

Now, we assume as our induction step that if $n = 2^k$ for $k > 1$,

$$T(n) = 2T(n/2) + n = n \lg n.$$

Then, suppose that $n = 2^{k+1}$, so that we have

$$\begin{aligned}
T(n) &= 2T(2^{k+1}/2) + 2^{k+1} \\
&= 2T(2^k) + 2^{k+1} \\
&= 2(2^k \lg 2^k) + 2^{k+1} && \text{(by our induction assumption)} \\
&= 2^{k+1} \lg 2^k + 2^{k+1} \lg 2 && \text{(since } \log_2 2 = 1\text{)} \\
&= 2^{k+1} \lg 2^{k+1} && \text{(since } \log_2 a + \log_2 b = \log_2 ab\text{)}
\end{aligned}$$

Therefore, by induction on $n = 2^k$, $T(n) = n \lg n$. □

2.3-4

We let $T(n)$ be the running time of a problem of size n. For $n \leq 2$, we have $T(n) = \Theta(2) = \Theta(1)$, since this is just the process of inserting an item in either position 1 or 2, so a worst-case time is $\Theta(1)$.

If $n > 2$, our division of the problem yields $n-1$ subproblems, each of which is $\frac{n-1}{n}$ times the size of the original. It takes $T(\frac{n(n-1)}{n}) = T(n-1)$ to solve one subproblem, so it takes $(n-1)T(n-1)$ to solve $n-1$ of them. It then takes $\Theta(n)$ time to divide the problem into subproblems and insert the last item in the array into $A[1\,..\,n-1]$, so we have

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 2 \\ (n-1)T(n-1) + \Theta(1) & \text{otherwise} \end{cases}.$$

2.3-5

BINARY-SEARCH(A, ν)

```
1: low = 1
2: high = A.length
3: while low < high do
4:     mid = ⌊(high+low)/2⌋
5:     if A[mid] == ν then
6:         return mid
7:     else if A[mid] > ν then
8:         high = mid − 1
9:     else
```

10: $low = mid + 1$
11: **return** -1

Suppose we have an array A of length n. Then, in the worst case, we loop through the **while** loop of BINARY-SEARCH (lines 3-9) until $low = high$ and return -1, so that ν is not in A. If we continually divide n by 2, and n is a power of 2, then we will execute

$$\lfloor \frac{high - low}{2} \rfloor$$

exactly $\lg n$ times. If n is not a power of 2, then the same operation executes $\lfloor \lg n \rfloor$ times. In any case, we have a worst-case running time of $\Theta(\lg n)$ for BINARY-SEARCH.

2.3-6

Since the loop invariant of INSERTION-SORT guarantees that we have $A[1..j-1]$ sorted, we can use the BINARY-SEARCH algorithm of 2.3-5 in place of the **while** loop of lines 5-7.

Since every other operation in the algorithm for INSERTION-SORT is $\Theta(n)$ and the **while** loop is nested but also $\Theta(n)$, we can improve the overall worst-case running time to $\Theta(n \lg n)$, which is better than $\Theta(n^2)$ for the original INSERTION-SORT algorithm.

2.3-7

Suppose we take the recursive approach of dividing the set S of n integers in half and checking each half. If the first half has index 1 and index $\lfloor n/2 \rfloor$ referencing elements such that for $A = S$ enumerated as an array, $A[1] = A[\lfloor n/2 \rfloor] = x$, we return *true*. If not, we check the other half with the same condition and if the condition is *false* for the upper half, wle recursively check each lower and upper half of the two subarrays. This process of repeated recursive subdivisions will have a running time of $\Theta(n \lg n)$, since the recurrence is the same as for merge sort.

Problems

2-1

a. *Proof.* Since INSERTION-SORT has a running time of $\Theta(n^2)$, due to the nested **while** loop of lines 5-7 (p. 26), and the loop of lines 1-7, if we are instead sorting n items divided into n/k sublists, the outer **for** loop will run k times for each sublist. Since each sublist will contain k items, the **while** loop will run $k - 1$ times for each sublist, so we have a running time of $\Theta(k^2)$ for each sublist, and since there are n/k sublists, the total running time will be

$$\Theta\left(\frac{n}{k} \cdot k^2\right) = \Theta(nk).$$

□

b. If we merge the sublists when their length reaches k, we are removing $\lg k$ levels from the recursion tree on p. 38, so we have

$$\lg n - \lg k = \lg n/k$$

levels on the tree. Now, from **a.**, we have the INSERTION-SORT algorithm on the n/k sublists with a running time of $\Theta(nk)$, so the new algorithm will have a running time of

$$\Theta(n \lg (n/k) + nk) = \Theta(n[\lg (n/k) + k]) = \Theta(n \lg (n/k)).$$

c. We are looking for the point at which

$$\begin{aligned} \Theta(nk + n \lg (n/k)) &= \Theta(n \lg n) \\ \Leftrightarrow c_1[nk + n \lg (n/k)] &= c_2 n \lg n \\ \Leftrightarrow c_2 \lg n &= c_1[k - \lg n/k] \\ \Leftrightarrow k + \lg k &= \frac{(c_1 + c_2) \lg n}{c_2}. \end{aligned}$$

d. In practice, we should choose k to be $\lceil \lg n \rceil$, since this will be the closest point to which

$$k + \lg k = \frac{(c_1 + c_2) \lg n}{c_2},$$

where $k \in \mathbb{Z}$.

2-2

a. We need to prove a loop invariant for each loop in the algorithm.

b. *Proof.* At the start of each iteration of the **for** loop of lines 2-4, $A'[1 \,..\, i]$ consists of elements sorted in ascending order.

Initialisation: For the first loop of lines 2-4, $A'[1 \,..\, i]$ is a one-element array, $A'[1]$, which is (trivially) sorted.

Maintenance: After each **for** loop, the first out-of-order element from $A.length$ down to $i + 1$ has been swapped with each element before it until it is in the correct position in $A'[1 \,..\, i]$.

Termination: The **for** loop terminates when $j == i + 1$, so that, at that point, $A'[i] < A'[i + 1]$ and the loop invariant holds for $A'[1 \,..\, i + 1]$.

□

c. *Proof.* For each iteration of the **for** loop of lines 1-4, $A'[1 \,..\, i + 1]$ is sorted in ascending order.

Initialisation: At initialisation, $i = 1$, so $A'[1 \,..\, i + 1]$ is $A'[1 \,..\, 2]$ and the termination of **b.** guarantees that this will be sorted.

Maintenance: Each execution of the **for** loop of lines 2-4 sorts $A'[1\,..\,i+1]$, so the loop invariant is maintained.

Termination: The loop terminates when $i == A.length - 1$, so $A'[1\,..\,i+1] = A'[1\,..\,n]$ has been sorted.

□

d. The worst-case running time occurs when A is sorted in descending order, so the **for** loop of lines 1-4 executes $n-1$ times and the **for** loop of lines 2-4 executes $n-i+1$ times. Thus we have $(n-1)(n-i+1) = \Theta(n^2)$ worst-case running time, equivalent to that of INSERTION-SORT.

2-3

a. The running time is $\Theta(n)$, since there is only one **for** loop that runs n times.

b. HORNERS-RULE$(a_1\,..\,a_n, n, x)$

```
1: y = 0
2: for i = n downto 0 do
3:     temp = x
4:     for j = i downto 0 do
5:         temp = temp · x
6:     y = a_i · temp + y
7: return y
```

The running time of this algorithm is $\Theta(n^2)$, since the outer **for** loop of lines 2-6 runs n times and for each execution of this loop, the inner **for** loop of lines 4-5 runs $n-i$ times, where i is the value of the loop iteration variable declared in line 2 of that iteration.

Cleary this algorithm has a much greater running time than Horner's rule.

c. *Proof.* **Initialisation:** The initialisation of the **for** loop of lines 2-3 assigns i to n, thus

$$\sum_{k=0}^{n-(i+1)} a_{k+i+1}x^k = \sum_{k=0}^{n-(n+1)} a_{k+n+1}x^k = \sum_{k=0}^{-1} a_{k+n+1}x^k = 0,$$

which is valid, since $y = 0$ upon initialisation of the **for** loop, therefore its initialisation is correct.

Maintenance: For each subsequent iteration of the **for** loop, y is assigned to $a_i + x \cdot y$, so that, for the first **for** loop, y is assigned to $a_1 + x \cdot 0 = a_1$, and

$$\sum_{k=0}^{n-(n-1+1)} a_{k+(n-1)+1}x^k = \sum_{k=0}^{0} a_{k+n}x^k = a_1,$$

which is correct.

For subsequent **for** loops, $a_i + x \cdot y$ multiplies the value of the previous **for** loop,

$$\sum_{k=0}^{n-(i+2)} a_{k+i+2} x^k$$

by x, so that we have

$$x \sum_{k=0}^{n-i-2} a_{k+i+2} x^k = \sum_{k=0}^{n-i-2} a_{k+i+2} x^{k+1},$$

and we add a_i to this value, with the result

$$y = \sum_{k=0}^{n-(i+1)} a_{k+i} x^{k+1}.$$

Thus the loop invariant is maintained at each iteration.

Termination: Since the **for** loop terminates at $i = 0$, we have

$$\sum_{k=0}^{n-(i+1)} a_{k+i+1} x^{k-1} = \sum_{k=0}^{n-1} a_{k+1} x^{k-1}$$

at the beginning of the lop, and with one more operation,

$$y = \sum_{k=0}^{n} a_k x^k,$$

so that the loop terminates correctly.

□

d. Since the loop invariant is initialised, maintained, and terminates correctly, for

$$P(x) = \sum_{k=0}^{n} a_k x^k,$$

the correct evaluation is performed for $a_0, a_1, \ldots, a_n$.

2-4

a. $(1, 5), (2, 5), (3, 5), (4, 5),$ and $(3, 4)$.

b. The array containing all elements of the set $\{1, 2, \ldots, n\}$ in reverse order has the most possible inversions. Since there are $n - 1$ inversions, starting with 1, $n - 2$ starting with 2, ..., there are

$$\sum_{k=1}^{n-1} k$$

inversions, or

$$\frac{n(n-1)}{2}$$

inversions.

c. Since the worst-case running time of INSERTION-SORT occurs for exactly the array discussed in **b.** above, and we know that INSERTION-SORT has a worst-case running time which is $\Theta(n^2)$, we see that there is a direct correspondence between the number of inversions in the input array of INSERTION-SORT and its running time. This follows logically from the fact that each element from $j = 2$ to $A.length$ (where A is the input array) must be sorted if there exists an inversion with j as its second coordinate.

d. As with MERGE-SORT, we have two subalgorithms, one which finds the number of inversions in a given subarray as follows:

MERGE-INVERSIONS(A, p, q, r)

```
 1: n_1 = q - p + 1
 2: n_2 = r - q
 3: let L[1..n_1 + 1] and R[1..n_2 + 1] be new arrays
 4: for i = 1 to n_1 do
 5:     L[i] = A[p + i - 1]
 6: for j = 1 to n_2 do
 7:     R[j] = A[q + j]
 8: L[n_1 + 1] = ∞
 9: R[n_2 + 1] = ∞
10: i = 1
11: j = 1
12: inversions = 0
13: counted = FALSE
14: for k = p to r do
15:     if counted == FALSE and R[j] < L[i] then
16:         inversions = inversions + n_1 - i + 1
17:         counted = TRUE
18:     if L[i] ≤ R[j] then
19:         A[k] = L[i]
20:         i = i + 1
21:     else
22:         A[k] = R[j]
23:         j = j + 1
24:         counted = FALSE
25: return inversions
```

Now that we have the recursive procedure that separates the array into subarrays and finds the number of inversions in each, we find the total number of inversions using the following algorithms:

COUNT-INVERSIONS$(A, p, r, inversions)$

```
1: if p < r then
2:     q = ⌊(p + r)/2⌋
3:     inversions = inversions +
           COUNT-INVERSIONS(A, p, q, inversions)
4:     inversions = inversions +
           COUNT-INVERSIONS(A, q + 1, r, inversions)
5:     inversions = MERGE-INVERSIONS(A, p, q, r) + inverions
6: return inversions
```

Since the algorithm is based entirely upon MERGE-SORT, with almost the same number of steps, it also has $\Theta(n \lg n)$ running time.

3 Growth of Functions

3.1 Asymptotic notation

3.1-1

Proof. Suppose $\max(f(n), g(n)) = f(n)$, then $f(n) \geq g(n)$ for all $n > n_0$, for some $n_0 \in \mathbb{R}^+$. Then, since the Θ-relation is reflexive, $f(n) = \Theta(f(n))$, so we have some $c_1, c_2, n_0 \in \mathbb{R}^+$ such that $0 \leq c_1 f(n) \leq f(n) \leq c_2 f(n)$ for all $n > n_0$.

Now, since $g(n)$ is asymptotically positive, it follows that $f(n) \leq c_2 f(n) \leq c_2 f(n) + c_2 g(n) = c_2(f(n) + g(n))$. Then, $f(n) = \max(g(n), f(n)) \Rightarrow c_3(f(n) + g(n)) \leq c_1 f(n)$ for some $c_3 \in \mathbb{R}^+$, provided c_3 is sufficiently small and $c_3 < c_1$.

Thus, we have $0 \leq c_3(f(n), g(n)) \leq \max(f(n), g(n)) \leq c_2(f(n) + g(n))$, and we have $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

A similar argument proves the case where $\max(f(n), g(n)) = g(n)$. □

3.1-2

Proof. By the binomial expansion, we have

$$(n + a)^b = \sum_{k=0}^{b} \binom{b}{k} n^k a^{b-k},$$

and since $b \in \mathbb{R}$ and $b > 0$, it follows that $0 \leq k \leq b$ for all terms in the expansion, so that the highest-order term is

$$n^b a^{b-b} = n^b a^0 = n^b.$$

Thus $(n + a)^b = \Theta(n^b)$. □

3.1-3

Since big-O notation gives an upper bound on the running time of a function, it does not make sense to say "at least" in reference to an O-class of functions, as "at least" implies that we are referring to a lower bound.

3.1-4

Since $2^{n+1} = 2 \cdot 2^n$, i.e., for $g(n) = 2^{n+1}$ and $f(n) = 2^n$, we have $g(n) = c \cdot f(n)$ for all $n_0 > 0$, where $c = 2$ and $n_0 = 0$, we do have $2^{n+1} = O(2^n)$.

However, $2^{2n} = 2^n \cdot 2^n$, so we do not ahve any $c \in \mathbb{R}^+$ such that $g(n) = 2^{2n} = c \cdot f(n)$ for $n \geq n_0$, where $n, n_0 \in \mathbb{N}$, $f(n) = 2^n$. Thus $2^{2n} \neq O(2^n)$.

3.1-5

Proof. ($\Rightarrow$) Suppose $f(n) = \Theta(g(n))$, then we have $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$, where $c_1, c_2 \in \mathbb{R}^+, n, n_0 \in \mathbb{N}$.

But then it follows that $0 \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$, so $f(n) = O(g(n))$, and $0 \leq c_1 g(n) \leq f(n)$ for all $n \geq n_0$, so $f(n) = \Omega(g(n))$.

($\Leftarrow$) Now, suppose $f(n) = O(g(n))$, i.e., $0 \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$ and $f(n) = \Omega(g(n))$, i.e., $0 \leq c_1 g(n) \leq f(n)$ for all $n \geq n_0$, where $c_1, c_2 \in \mathbb{R}^+, n, n_0 \in \mathbb{N}$. But then we have $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$, so that $f(n) = \Theta(g(n))$.

□

3.1-6

Proof. ($\Rightarrow$) Suppose the running time of an algorithm is $\Theta(g(n))$, i.e., $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$, where $c_1, c_2 \in \mathbb{R}^+, n, n_0 \in \mathbb{N}$ and $f(n)$ is a function that describes the running time of the algorithm in question.

Now, clearly the worst-case running time of the algorithm is some value $M \leq c_2 g(n)$, since $x = f(n)$ for the greatest possible value of M. But then we have $0 \leq x \leq c_2 g(n)$ and the running time $M = O(g(n))$.

Likewise, if x' is the best-case running time and m is the minimum value of $f(n)$ for all $n \in \mathbb{N}$ and $0 \leq c_1 g(n) \leq m$, so $m = \Omega(g(n))$.

($\Leftarrow$) Now, suppose the worst-case running time $M = O(g(n))$ and the best-case running time $m = \Omega(g(n))$. Then we have $0 \leq c_1 g(n) \leq m$ and $0 \leq M \leq c_2 g(n)$ for all $n \geq n_0$, where $c_1, c_2 \in \mathbb{R}^+, n, n_0 \in \mathbb{N}$. But clearly $m \leq f(n) \leq M$ for all n, so that it follows that

$$0 \leq c_1 g(n) \leq m \leq g(n) \leq M \leq c_2 g(n)$$

for all $n > n_0$, i.e., $f(n) = \Theta(g(n))$.

□

3.1-7

Proof. Suppose we have some function $f \in \{o(g(n)) \cap \omega(g(n))\}$. Then we have, for any $c_1, c_2 \in \mathbb{R}^+$, that there exists $n_0 > 0$ such that $0 \leq f(n) < c_2 g(n)$ and $0 \leq c_1 g(n) < f(n)$ for all $n \geq n_0$.

But now we have

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

and

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty,$$

so that we have a contradiction. Thus, we conclude that such an f does not exist, and $o(g(n)) \cap \omega(g(n)) = \varnothing$, the empty set. □

3.1-8

$$\begin{aligned}\Omega(g(n,m)) = \{f(n,m) : &\text{ there exist positive constants } c, n_0, \text{ and } m_0\\ &\text{such that } 0 \le cg(n,m) \le f(n,m)\\ &\text{for all } n \ge n_0 \text{ or } m \ge m_0\}\end{aligned}$$

and

$$\begin{aligned}\Theta(g(n,m)) = \{f(n,m) : &\text{ there exist positive constants } c_1, c_2, n_0, \text{ and } m_0\\ &\text{such that } 0 \le c_1 g(n,m) \le f(n,m) \le c_2 g(n,m)\\ &\text{for all } n \ge n_0 \text{ or } m \ge m_0\}.\end{aligned}$$

3.2 Standard notations and common functions

3.2-1

Proof. **a.** Suppose $f(n)$ and $g(n)$ are monotonically increasing functions, i.e., $m \le n \Rightarrow f(m) \le f(n)$ and $g(m) \le g(n)$. Then, clearly for $m \le n$, we have

$$f(m) + g(m) \le f(n) + g(m) \le f(n) + g(n),$$

so that $f(n) + g(n)$ is monotonically increasing.

b. Since $m \le n \Rightarrow g(m) \le g(n)$ and $f(m) \le f(n)$, it must also be the case that

$$m \le n \Rightarrow g(m) \le g(n) \Rightarrow f(g(m)) \le f(g(n)),$$

so that $f(g(n))$ is a monotonically increasing function.

c. Suppose further that $f(n)$ and $g(n)$ are nonnegative. Then

$$m \le n \Rightarrow f(m) \le f(n) \Rightarrow f(m) \cdot g(n) \le f(n) \cdot g(n)$$

and

$$m \le n \Rightarrow g(m) \le g(n) \Rightarrow f(n) \cdot g(m) \le f(n) \cdot g(n),$$

so that we have

$$m \le n \Rightarrow f(m) \cdot g(m) \le f(n) \cdot g(n),$$

and $f(n) \cdot g(n)$ is a monotonically increasing function.

□

3.2-2

Proof. For all $a, b, c > 0 \in \mathbb{R}$, we have

$$\begin{aligned}\log_b c &= \frac{\log_a c}{\log_a b} \\ &= \log_b a \cdot \log_a c \\ &= \log_a c \cdot \log_b a.\end{aligned}$$

Thus we have

$$\begin{aligned}a^{\log_b c} &= a^{\log_a c \cdot \log_b a} \\ &= \left(a^{log_a c}\right)^{\log_b a} \\ &= c^{\log_b a}.\end{aligned}$$

□

3.2-3

Proof. Since $n! \leq n^n$ for all $n \geq 1$, we have $\lg(n!) \leq \lg n^n = n \lg n$ for all $n \geq 1$, so that we have $\lg(n!) \leq c_2 \cdot (n \lg n)$ for $n \geq n_0$ with the identifications $c_2 = 1$ and $n_0 = 1$. Now, from Stirling's approximation, we have

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right) = n^{n+1/2}\frac{\sqrt{2\pi}}{e^n}\left(1 + \Theta\left(\frac{1}{n}\right)\right),$$

so that

$$\begin{aligned}\lg(n!) &\geq \lg n^{n+1/2} + \lg\left(\frac{\sqrt{2\pi}}{e^n}\right) \\ &= \left(n + \frac{1}{2}\right)\lg n + \lg\sqrt{2\pi} - n\lg e \\ &= n\lg n + \frac{1}{2}\lg n + \lg\sqrt{2\pi} - n\lg e.\end{aligned}$$

Then, $\lg(n!) \geq n \lg n$ for

$$\begin{aligned}\frac{1}{2}\lg n + \lg\sqrt{2\pi} - n\lg e &\geq 0 \\ \Rightarrow n &\geq \frac{\lg\sqrt{2\pi n}}{\lg e} \\ \Rightarrow n - \frac{\lg n}{2\lg e} &\geq \frac{\lg\sqrt{2\pi}}{\lg e} \\ \Rightarrow n - \frac{1}{2}\ln n &\geq \ln\sqrt{2\pi} \\ \Rightarrow n &\geq \ln\sqrt{2\pi n}.\end{aligned}$$

Thus $\lg(n!) = \Theta(n \lg n)$.

Now,

$$n! = n \cdot (n-1) \cdots 3 \cdot 2 \cdot 1 < n \cdot n \cdots n \quad (n \text{ factors}),$$

for all $n > 1$, so that we have

$$n! = o(n \cdot n \cdots n) = o(n^n).$$

Then,

$$n! = n \cdot (n-1) \cdots 3 \cdot 2 \cdot 1 > 2 \cdot 2 \cdots 2 \quad (n \text{ factors}),$$

for all $n > 2$, so that we have

$$n! = \omega(2 \cdot 2 \cdots 2) = \omega(2^n).$$

□

4 Divide-and-Conquer

4.1 The maximum-subarray problem

4.1-1

It still returns the maximum subarray, the use of negative values does not affect the comparison criteria, as we are still comparing differences between successive elements.

4.1-2

BRUTE-FORCE-MAXIMUM-SUBARRAY(A)

```
1:  low = 0
2:  high = 1
3:  sum = 0
4:  maxSum = 0
5:  first = low
6:  last = high
7:  while low < A.length − 1 do
8:    while high < A.length do
9:      sum = sum + A[high] − A[low]
10:     if sum > maxSum then
11:       maxSum = sum
12:       first = low
13:       last = high
14:     high = high + 1
15:   low = low + 1
16:   high = low + 1
17: return (first, last, maxSum)
```

4.1-3

For an array of size $n = 20$, the divide-and-conquer algorithm overtakes the brute-force algorithm, using a Java implementation of both algorithms. The hybrid approach using brute-force for $n_0 < 20$ seems to run faster than the purely recursive approach for all values of n_0.

4.1-4

In order to account for an empty subarray, we need only specify that *left-sum*, *right-sum*, or *cross-sum* be greater than 0 in the conditional **if** ... **else** statement of our algorithms. We may then add a final **else** clause allowing us to return an empty subarray if none of the other conditionals return a true value.

4.1-5

```
FIND-MAXIMUM-SUBARRAY(A, low, high)
 1: j = low + 1
 2: maxLow = 1
 3: maxHigh = 2
 4: maxValue = A[low] + A[j]
 5: currentValue = maxValue
 6: while j < high do
 7:    currentValue = A[j + 1]
 8:    for i = j downto low do
 9:       currentValue = currentValue + A[i]
10:       if maxValue < currentValue then
11:          maxLow = i
12:          maxHigh = j + 1
13:          maxValue = currentValue
14:    j = j + 1
15: return (maxLow, maxHigh, maxValue)
```

4.2 Strassen's algorithm for matrix multiplication

4.2-1

For the matrix product

$$\begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix} \begin{pmatrix} 6 & 8 \\ 4 & 2 \end{pmatrix},$$

we assign

$$A = \begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix}$$

and

$$B = \begin{pmatrix} 6 & 8 \\ 4 & 2 \end{pmatrix},$$

so we have

$$\begin{aligned}
S_1 &= B_{12} - B_{22} = 8 - 2 = 6, \\
S_2 &= A_{11} + A_{12} = 1 + 3 = 4, \\
S_3 &= A_{21} + A_{22} = 7 + 5 = 12, \\
S_4 &= B_{21} - B_{11} = 4 - 6 = -2, \\
S_5 &= A_{11} + A_{22} = 1 + 5 = 6, \\
S_6 &= B_{11} + B_{22} = 6 + 2 = 8, \\
S_7 &= A_{12} - A_{22} = 3 - 5 = -2, \\
S_8 &= B_{21} + B_{22} = 4 + 2 = 6, \\
S_9 &= A_{11} - A_{21} = 1 - 7 = -6, \\
\text{and} \quad S_{10} &= B_{11} + B_{12} = 6 + 8 = 14.
\end{aligned}$$

Then, we recursively multiply the $n/2 \times n/2 = 1 \times 1$ matrices seven times to compute

$$\begin{aligned}
P_1 &= A_{11} \cdot S_1 = 1 \cdot 6 = 6, \\
P_2 &= S_2 \cdot B_{22} = 4 \cdot 2 = 8, \\
P_3 &= S_3 \cdot B_{11} = 12 \cdot 6 = 72, \\
P_4 &= A_{22} \cdot S_4 = 5 \cdot (-2) = -10, \\
P_5 &= S_5 \cdot S_6 = 6 \cdot 8 = 48, \\
P_6 &= S_7 \cdot S_8 = -2 \cdot 6 = -12, \\
\text{and} \quad P_7 &= S_9 \cdot S_{10} = -6 \cdot 14 = -84.
\end{aligned}$$

Now, we add and subtract the P_i submatrices to obtain

$$\begin{aligned}
C_{11} &= P_5 + P_4 - P_2 + P_6 = 48 + (-10) - 8 + (-12) = 18, \\
C_{12} &= P_1 + P_2 = 6 + 8 = 14, \\
C_{21} &= P_3 + P_4 = 72 + (-10) = 62, \\
\text{and} \quad C_{22} &= P_5 + P_1 - P_3 - P_7 = 48 + 6 - 72 - (-84) = 66.
\end{aligned}$$

Thus, we have

$$C = A \cdot B = \begin{pmatrix} 18 & 14 \\ 62 & 66 \end{pmatrix}.$$

4.2-2

STRASSEN-MULTIPLY(A, B)
1: $n = A.rows$
2: let C be a new $n \times n$ matrix
3: **if** $n == 1$ **then**
4: $\quad C_{11} = A_{11} \cdot B_{11}$
5: $\quad$ **return** C

6: **else**
7: partition A, B, and C as in equations (4.9)
8: $S_1 = B_{12} - B_{22}$
9: $S_2 = A_{11} + A_{12}$
10: $S_3 = A_{21} + A_{22}$
11: $S_4 = B_{21} - B_{11}$
12: $S_5 = A_{11} + A_{22}$
13: $S_6 = B_{11} + B_{22}$
14: $S_7 = A_{12} - A_{22}$
15: $S_8 = B_{21} + B_{22}$
16: $S_9 = A_{11} - A_{21}$
17: $S_{10} = B_{11} + B_{12}$
18: $P_1 = \text{STRASSEN-MULTIPLY}(A_{11}, S_1)$
19: $P_2 = \text{STRASSEN-MULTIPLY}(S_2, B_{22})$
20: $P_3 = \text{STRASSEN-MULTIPLY}(S_3, B_{11})$
21: $P_4 = \text{STRASSEN-MULTIPLY}(A_{22}, S_4)$
22: $P_5 = \text{STRASSEN-MULTIPLY}(S_5, S_6)$
23: $P_6 = \text{STRASSEN-MULTIPLY}(S_7, S_8)$
24: $P_7 = \text{STRASSEN-MULTIPLY}(S_9, S_{10})$
25: $C_{11} = P_5 + P_4 - P_2 + P_6$
26: $C_{12} = P_1 + P_2$
27: $C_{21} = P_3 + P_4$
28: $C_{22} = P_5 + P_1 - P_3 - P_7$
29: **return** C

4.2-3

We have the following algorithm:

STRASSEN-MULTIPLY(A, B)
1: $n = A.rows$
2: let C be a new $n \times n$ matrix
3: **if** $n == 1$ **then**
4: $C_{11} = A_{11} \cdot B_{11}$
5: **return** C
6: **else if** $2 \nmid n$ **then**
7: **for** $i = 1$ **to** n **do**
8: $c_{i1} = a_{i1} \cdot b_{1i}$
9: **for** $j = 1$ **to** n **do**
10: $c_{1j} = a_{1j} \cdot b_{j1}$
11: partition A, B, and C by excluding top row and left column of each into A_{lower}, B_{lower}, and C_{lower}, respectively
12: $C_{lower} = \text{STRASSEN-MULTIPLY}(A_{lower}, B_{lower})$
13: **else**
14: partition A, B, and C as in equations (4.9)
15: $S_1 = B_{12} - B_{22}$
16: $S_2 = A_{11} + A_{12}$

17: $S_3 = A_{21} + A_{22}$
18: $S_4 = B_{21} - B_{11}$
19: $S_5 = A_{11} + A_{22}$
20: $S_6 = B_{11} + B_{22}$
21: $S_7 = A_{12} - A_{22}$
22: $S_8 = B_{21} + B_{22}$
23: $S_9 = A_{11} - A_{21}$
24: $S_{10} = B_{11} + B_{12}$
25: $P_1 = \text{STRASSEN-MULTIPLY}(A_{11}, S_1)$
26: $P_2 = \text{STRASSEN-MULTIPLY}(S_2, B_{22})$
27: $P_3 = \text{STRASSEN-MULTIPLY}(S_3, B_{11})$
28: $P_4 = \text{STRASSEN-MULTIPLY}(A_{22}, S_4)$
29: $P_5 = \text{STRASSEN-MULTIPLY}(S_5, S_6)$
30: $P_6 = \text{STRASSEN-MULTIPLY}(S_7, S_8)$
31: $P_7 = \text{STRASSEN-MULTIPLY}(S_9, S_{10})$
32: $C_{11} = P_5 + P_4 - P_2 + P_6$
33: $C_{12} = P_1 + P_2$
34: $C_{21} = P_3 + P_4$
35: $C_{22} = P_5 + P_1 - P_3 - P_7$
36: **return** C

The second case $2 \nmid n$ adds a single step which takes $2n$ time, so that the algorithm still runs in time $\Theta(n^{\lg 7} + 2n) = \Theta(n^{\lg 7})$, since the running time of STRASSEN-MULTIPLY is $\Theta(n^{\lg 7})$.

A Summations

A.1 Summation formulas and properties

A.1-1

$$\begin{aligned}\sum_{k=1}^{n}(2k-1) &= \sum_{k=1}^{n} 2k - \sum_{k=1}^{n} 1 \\ &= 2\sum_{k=1}^{n} k - \sum_{k=1}^{n} 1 \\ &= n(n+1) - n \\ &= n^2.\end{aligned}$$

A.1-2

Proof. We have

$$\begin{aligned}
\sum_{k=1}^{n} \frac{1}{2k-1} &= \sum_{k=1}^{n} \frac{1}{2} \cdot \frac{1}{k-\frac{1}{2}} \\
&= \frac{1}{2} \sum_{k=1}^{n} \frac{1}{k-\frac{1}{2}} \\
&= \frac{1}{2}\left[\ln\left(n-\frac{1}{2}\right) + O(1)\right] \\
&= \frac{1}{2}\ln\left(n-\frac{1}{2}\right) + O(1) \\
&= \frac{1}{2}\ln(n) + O(1) \\
&= \ln\sqrt{n} + O(1).
\end{aligned}$$

□

A.1-3

Proof. We have

$$\begin{aligned}
\frac{d}{dx}\sum_{k=0}^{\infty} kx^k &= \sum_{k=1}^{\infty} k^2 x^{k-1} \\
&= \frac{d}{dx}\frac{x}{(1-x)^2} \\
&= \frac{(1-x)^2 + 2x(1-x)}{(1-x)^4} \\
&= \frac{(1-x)+2x}{(1-x)^3} \\
&= \frac{x+1}{(1-x)^3}.
\end{aligned}$$

Thus we have

$$\begin{aligned}
x\sum_{k=0}^{\infty} k^2 x^{k-1} &= \sum_{k=0}^{\infty} k^2 x^k \\
&= \frac{x(x+1)}{(1-x)^3},
\end{aligned}$$

for $|x| < 1$. □

A.1-4

Proof. We have

$$\begin{aligned}\frac{k-1}{2^k} &= \frac{k}{2^k} - \frac{1}{2^k} \\ &= kx^k - x^k,\end{aligned}$$

for $x = \frac{1}{2}$. Now, since $|x| < 1$, from **A.6** and **A.8**, we have

$$\begin{aligned}\sum_{k=0}^{\infty}(k-1)/2^k &= \sum_{k=0}^{\infty} kx^k - \sum_{k=0}^{\infty} x^k \\ &= \frac{x}{(1-x)^2} - \frac{1}{1-x} \\ &= \frac{x-(1-x)}{(1-x)^2} \\ &= \frac{2x-1}{(1-x)^2} \\ &= \frac{1-1}{(-\frac{1}{2})^2} \\ &= 0.\end{aligned}$$

□

A.1-5

We have for $|x| < 1$,

$$\begin{aligned}\frac{d}{dx}\sum_{k=1}^{\infty}(2k+1) &= \sum_{k=1}^{\infty}(2k+1)x^{2k} \\ &= \frac{d}{dx}\frac{1}{1-x^{(2k+1)-k}} \\ &= \frac{d}{dx}\frac{1}{1-x^{k+1}} \\ &= \frac{(k+1)x^k}{(1-x^{k+1})^2}.\end{aligned}$$

A.1-6

Proof. We have

$$\begin{aligned}\sum_{k=1}^{n} O(f_k(i)) &= O(f_1(i)) + O(f_2(i)) + \cdots + O(f_n(i)) \\ &= \{f(i) : \text{ there exist positive constants } c \text{ and } i_0 \\ &\qquad \text{such that } 0 \le f(i) \le cf_1(i) \text{ for all } i \ge i_0\} \\ &\quad + \{f(j) : \text{ there exist positive constants } d \text{ and } j_0 \\ &\qquad \text{such that } 0 \le f(j) \le df_2(j) \text{ for all } j \ge j_0\} \\ &\quad + \cdots + \{f(w) : \text{ there exist positive constants } z \text{ and } w_0 \\ &\qquad \text{such that } 0 \le f(w) \le zf_n(w) \text{ for all } w \ge w_0\}.\end{aligned}$$

Interpreting summation in the above as set union, we have a superset ψ where $O(f_1(i)) \subseteq \psi, O(f_2(i)) \subseteq \psi, \ldots O(f_n(i)) \subseteq \psi$, since $\psi = \sum_{k=1}^{n} O(f_k(i))$.

Now, by the linearity property, $f_1(i) + f_2(i) + \cdots f_n(i) = \sum_{k=1}^{n} f_k(i)$ is in ψ, so it follows that $\psi = O\big(\sum_{k=1}^{n} f_k(i)\big)$. □